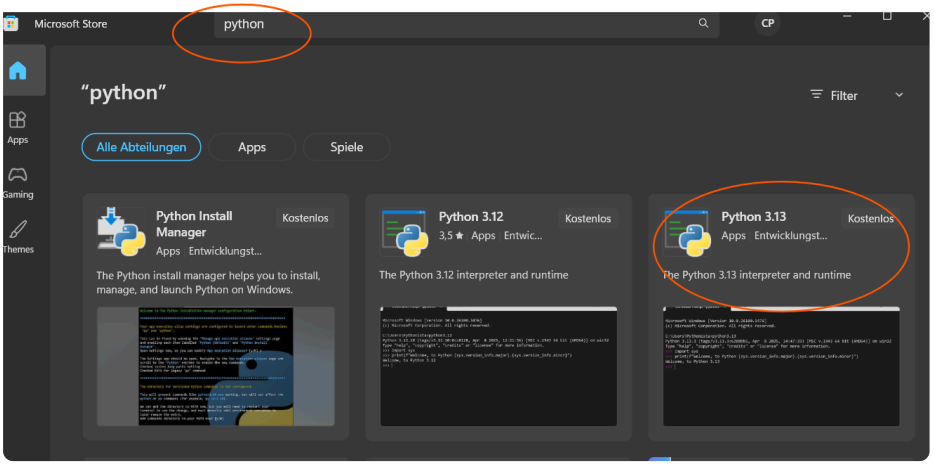
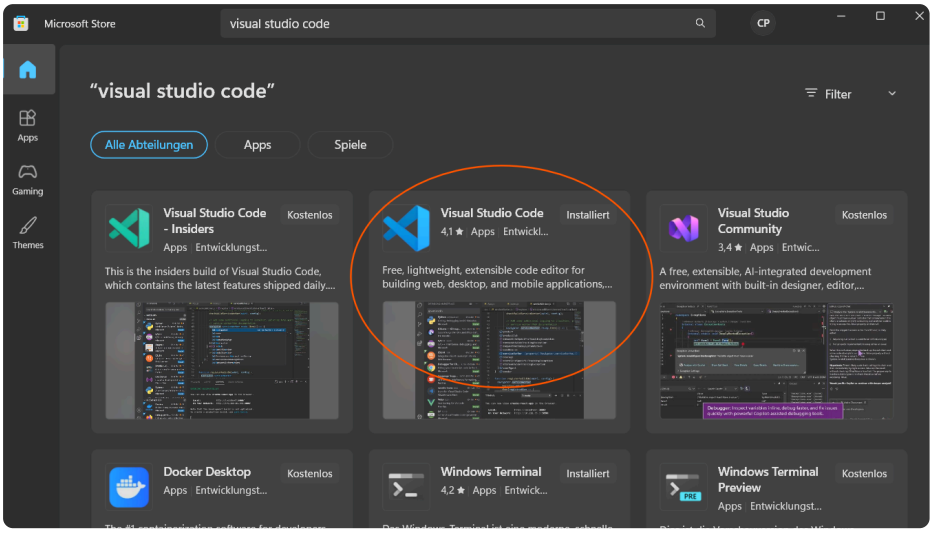
PRISM Kurzanleitung

1. Programme installieren und Repository klonen

1. Im Microsoft Store nach Python 3.13 suchen und installieren.

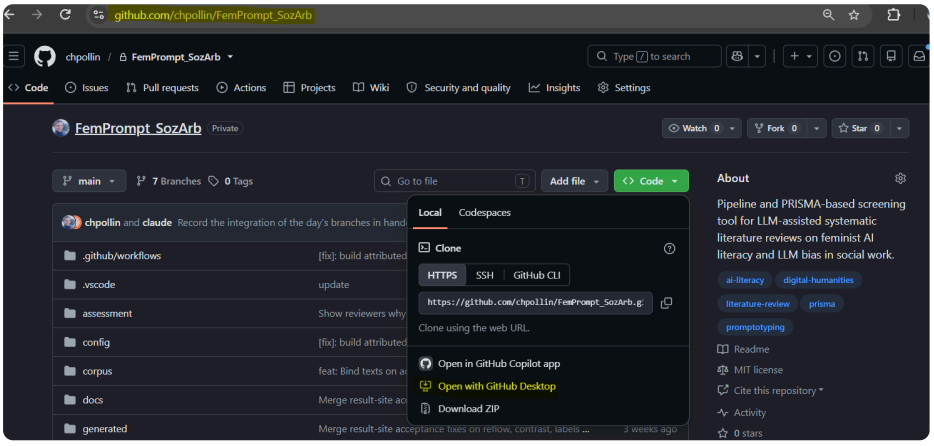


2. Im Microsoft Store nach Visual Studio Code suchen und installieren.



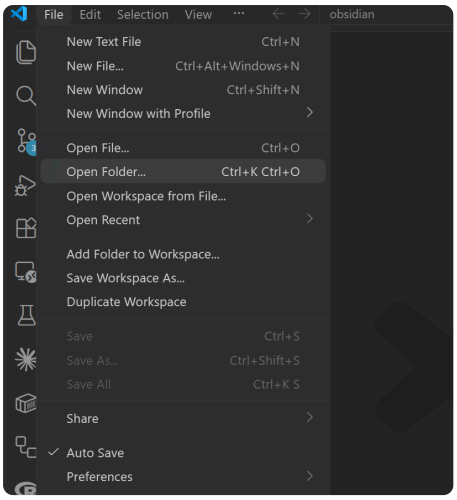
3. GitHub Desktop unter desktop.github.com/download installieren und unter File → Options → Accounts mit dem GitHub-Konto anmelden (optional Git für das Terminal unter git-scm.com/install/windows).

4. Im Browser github.com/chpollin/FemPrompt_SozArb öffnen, auf Code → Open with GitHub Desktop klicken und das Repository nach `Dokumente\GitHub\FemPrompt_SozArb` klonen.

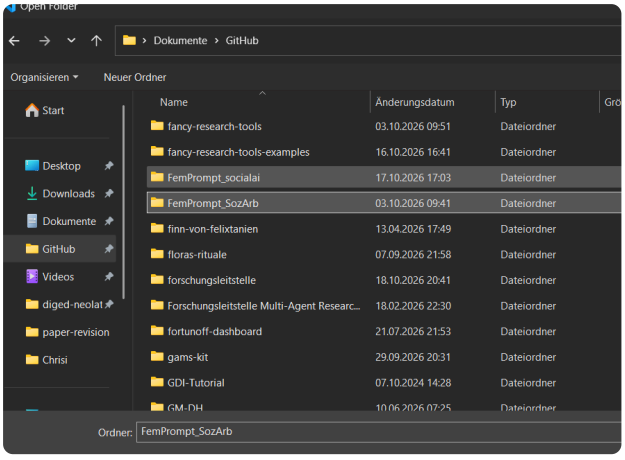


2. Projektordner und Terminal in Visual Studio Code öffnen

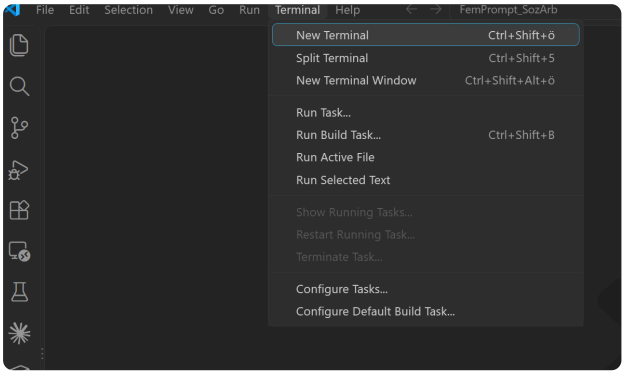
1. In Visual Studio Code im oberen Menü File → Open Folder wählen.



2. Den geklonten Ordner `Dokumente\GitHub\FemPrompt_SozArb` auswählen und öffnen.



3. Im oberen Menü Terminal → New Terminal (oder Strg + Shift + ö) wählen.



4. Einmalig beim ersten Einrichten die Python-Pakete im geöffneten Terminal installieren:

```
python --version
python -m pip install -r requirements-build.txt
```

3. PRISM starten

Im Terminal von Visual Studio Code (oder per Doppelklick im Windows Explorer) das Startskript ausführen:

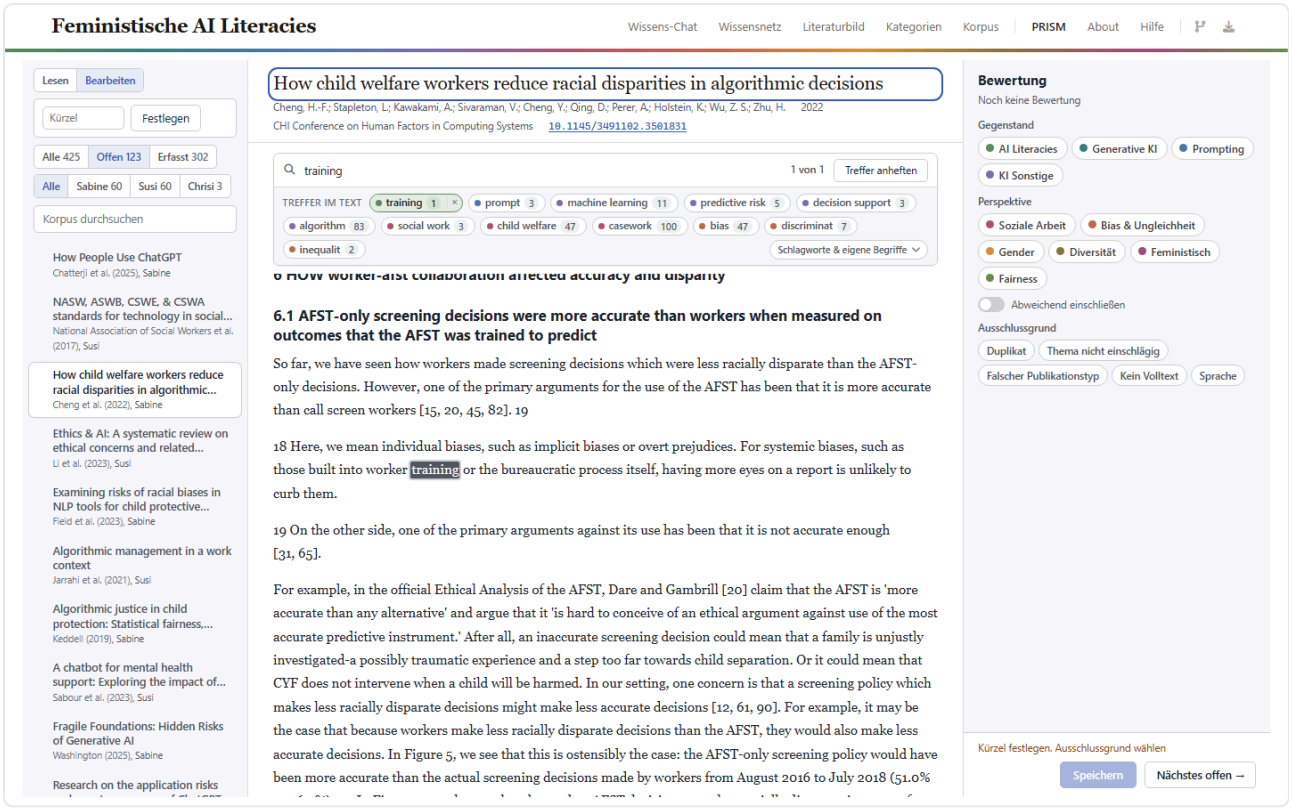
```
.\start-prism.cmd
```

Beim ersten Start baut das Skript lokal die Lesefassung der Volltexte auf und öffnet PRISM im Browser unter `http://127.0.0.1:8870/prisma.html?verify=1`. Das Terminalfenster während der Arbeit offen lassen und nach Ende der Sitzung mit Strg + C beenden.

4. Aufgabe und Arbeiten in PRISM

123 offene Werke werden ohne KI-Vorblendung im Volltext oder Abstract gescreent (Zotero-Dubletten zum selben Werk sind bereits auf den bevorzugten Haupteintrag zusammengeführt). Jede Person speichert in eine eigene Datei:

Profil in PRISM	Zieldatei im Repository	Zugeteilte offene Paper
Sabine	`docs/data/screening/sabine.json`	60 offene Paper
Susi	`docs/data/screening/susi.json`	60 offene Paper
Chrisi	`docs/data/screening/chrisi.json`	3 technische Paper (`978AV9KZ` , `RAY6G2R7` , `SQYLQFRU`)

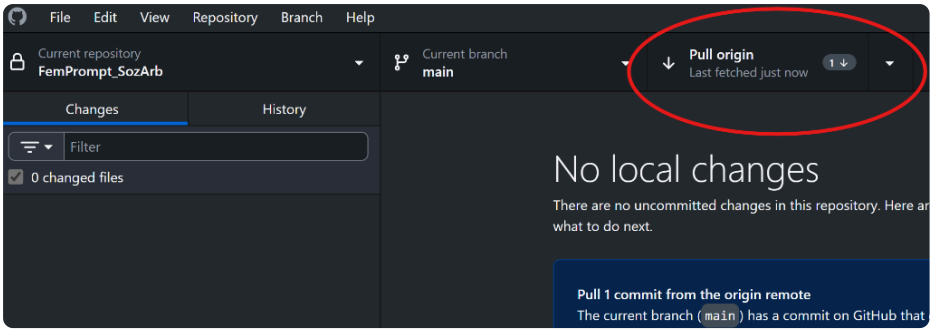


Ablauf beim Bewerten eines Papers in PRISM

1. Beim Öffnen von PRISM oben links den Modus Bearbeiten und das eigene Profil wählen (Sabine, Susi oder Chrisi). Der Filter Zuteilung und das erste Paper der eigenen Liste öffnen sich automatisch. Links den Filter Status auf Offen setzen.
2. Über dem Lesetext unterstützt die Volltextsuche das schnelle Prüfen von Begriffen. Unter dem Suchfeld erscheinen für alle zehn Kategorien sowie für zuletzt gesuchte Begriffe anklickbare Schlagwort-Chips, sobald ein Begriff im aktuellen Text vorkommt, jeweils mit seiner Trefferzahl im Dokument. Ein Klick auf einen Chip springt direkt zur ersten Fundstelle und hebt die Treffer in der Kategoriefarbe hervor. Über die Schaltfläche `Schlagworte & eigene Begriffe` lässt sich jeder eingetippte Suchbegriff einer der zehn Kategorien zuordnen (etwa für Prompting, Bias, Gender, Diversität oder Methode) und bleibt für alle weiteren Paper gespeichert.
3. Im rechten Panel Bewertung werden die zehn Kategorien in zwei Gruppen mit nein, teilweise oder ja bewertet:
   - Gruppe Gegenstand: AI Literacies, Generative KI, Prompting und KI Sonstige.
   - Gruppe Perspektive: Soziale Arbeit, Bias & Ungleichheit, Gender, Diversität, Feministisch und Fairness.
4. Include folgt automatisch, sobald mindestens eine Kategorie unter Gegenstand und mindestens eine Kategorie unter Perspektive auf ja stehen. Steht in beiden Gruppen mindestens eine Kategorie auf teilweise oder ja, ohne Include zu erreichen, lautet der Stand Unclear. Bleibt eine Gruppe ganz auf nein, folgt Exclude mit Auswahl eines Ausschlussgrunds (Duplikat, Thema nicht einschlägig, Falscher Publikationstyp, Kein Volltext oder Sprache). Über Abweichend einschließen ist bei Bedarf ein begründeter Override möglich.
5. Textbelege im Paper sind beim menschlichen Screening optional. Eine Bewertung lässt sich direkt über die Kategorien-Schalter und Speichern (oder Strg + S) sichern, auch ohne markierte Textstelle. Wer eine Entscheidung mit einer Textstelle untermauern möchte, markiert die Stelle mit der Maus im Papertext oder klickt in der Volltextsuche auf Treffer anheften und wählt die passende Kategorie im Auswahl-Dock.
6. Ein Klick auf Speichern (oder Strg + S) schreibt die Bewertung über den lokalen Server direkt in die eigene Datei `docs/data/screening/<kennung>.json`.

5. Stand mit GitHub Desktop synchronisieren

1. Vor jeder Sitzung in GitHub Desktop oben rechts auf Fetch origin klicken. Fetch prüft auf GitHub, ob neue Änderungen vorliegen. Zeigt die Schaltfläche danach Pull origin mit einer Zahl und Pfeil nach unten, einmal darauf klicken, um die neuen Änderungen auf den eigenen Rechner herunterzuladen.



2. Nach der Sitzung links unter Changes prüfen, dass ausschließlich die eigene Datei (`docs/data/screening/<kennung>.json`) geändert ist.

3. Unten links im Feld Summary eine kurze Commit-Nachricht mit vorangestelltem Namenskürzel eintragen, auf Commit to main und danach oben rechts auf Push origin klicken:

```
[sabine] 5 Paper bewertet
[susi] 5 Paper bewertet
[chrisi] 3 technische Paper bewertet
```